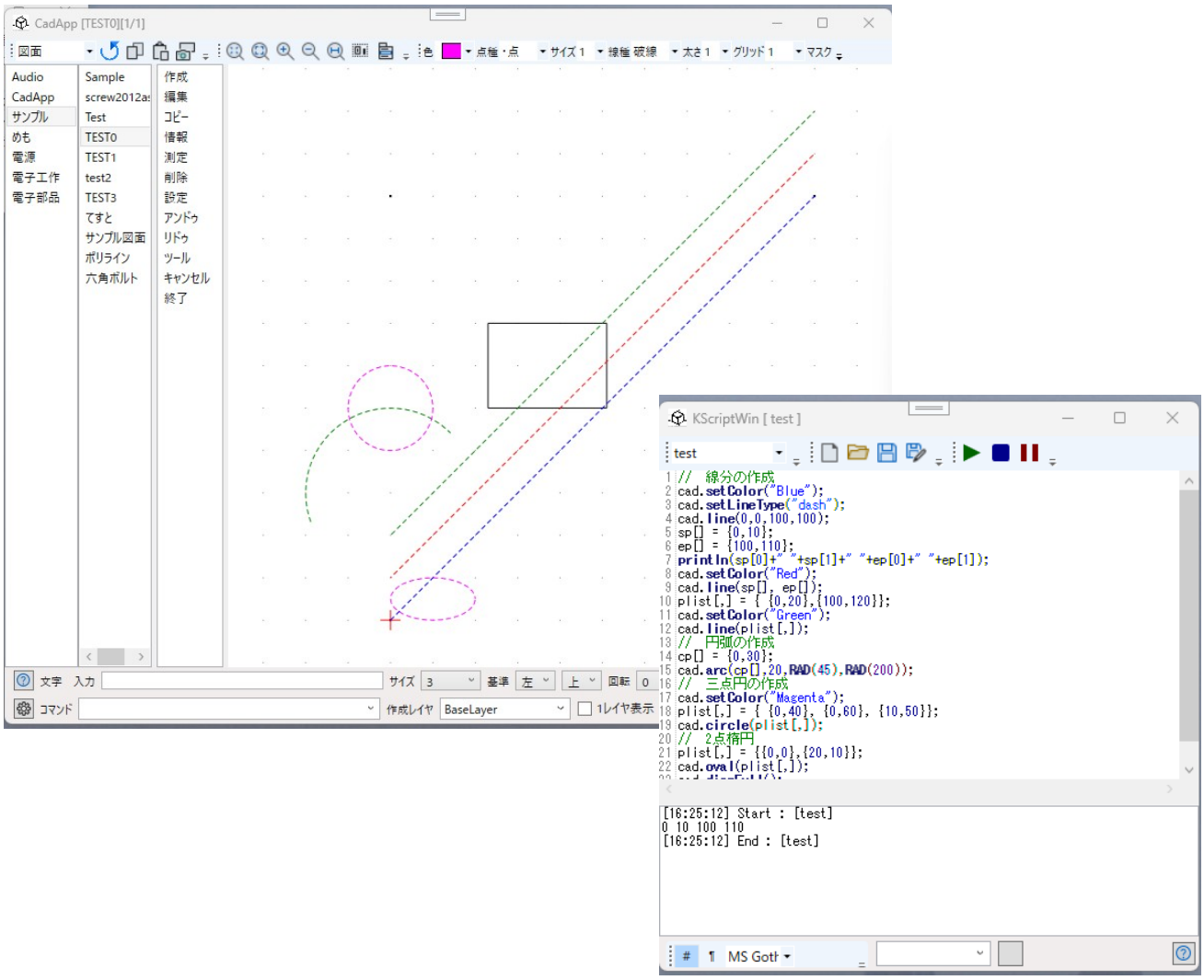
スクリプト処理

C 言語風のスクリプト言語とその実行環境

本 CAD で使用するスクリプト言語は C 言語に似せたスクリプト言語でインタプリタとして動作し、KScript と自称している。
C 言語風の制御構造と関数以外に図面作成のための要素作成や編集のための関数がある。
CAD コマンドの「ツール」→「スクリプト編集」でスクリプトの編集実行環境が表示れる。

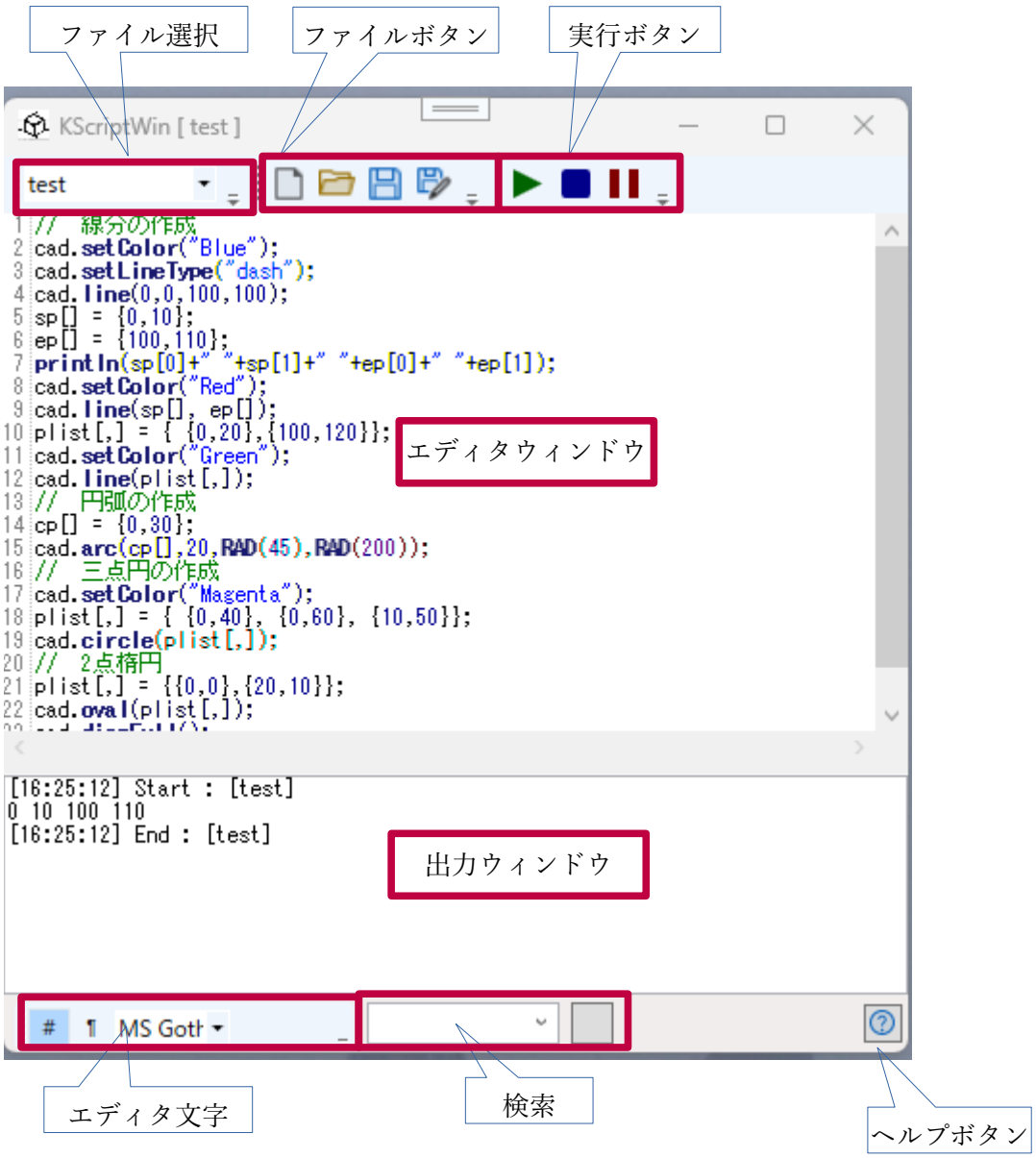


目次

1. 画面の説明.....4
2. プログラムの入力と実行.....6
プログラムの入力.....6
プログラムの実行.....6
プログラムの一時停止と中断.....6
ファイル参照.....6
入力候補.....7
3. スクリプト文.....8
4. スクリプト文の書き方.....9
スクリプト文の書き方.....9
コメント.....10
ローカル変数.....10
グローバル変数.....10
配列.....11
2 次元配列.....11
演算子.....12
複合演算子.....12
比較演算子.....12
制御文.....13
関数の定義.....15
ディレクティブ.....15
数式処理以外の関数.....15
ダイヤログ関数.....16
CAD 関数(cad.xxxx()).....16
5. 配列関数.....19
関数の種類.....19
6. マトリックス関数.....20
関数の種類.....20
7. CAD 機能.....21
設定関数.....21
要素作成関数.....21
線分の表示.....21
円弧の表示.....21

文字列の表示.....21

1. 画面の説明



・ファイル選択

カレントディレクトリのスクリプトファイルを選択してスクリプトをロード

・ファイルボタン

新規	エディタウィンドウをクリアし新規スクリプトの作成
開く	ファイル一覧からスクリプトファイルを読み込む
保存	編集したスクリプトをファイルに保存
名前を付けて保存	編集したスクリプトをファイル名を指定してに保存

・実行ボタン

実行	スクリプトの実行
中断	実行中のスクリプトの中断
一時停止	実行中のスクリプトの一時停止、再度押すと再開

・エディタウィンドウ　スクリプトを作成するウィンドウ
通常のエディタの機能以外に関数などの入力候補機能を追加

・出力ウィンドウ　print 文のメッセージを出力するウィンドウ

・エディタ文字

行番号(#)	行番号の表示/非表示
行末記号(¶)	行末記号の表示/非表示
文字フォント	エディタの文字フォントの選択
拡大	エディタの文字の拡大
縮小	エディタの文字の縮小

・検索　入力した文字列の検索

・キー入力操作

F5	プログラム実行
F8	入力候補
Apps(右 Win キー)	入力候補
Ctrl + A	全選択
Ctrl + C	コピー
Ctrl + D	一時停止
Ctrl + F	検索
Ctrl + N	新規
Ctrl + O	開く
Ctrl + S	上書き保存
Ctrl + V	貼付け
Ctrl + X	切取り
Ctrl + Y	リドゥ
Ctrl + Z	アンドゥ
Ctrl + ‘/’	コメント追加/解除
Shift + Alt + 矢印キー	矩形選択(解除は矢印キー単独など)
Shift + Ctrl + 上下矢印	エディタ画面上下移動(カーソル移動なし)

2. プログラムの入力と実行

プログラムの入力

プログラムの入力は左のエディタウィンドウから入力する。
エディタウィンドウの文字の種類や大きさは上ツールバーの左側のボタンで変更できる。
同じくツールバーの左側のボタンの中には行番号の有無、行末記号の有無の切替もできる。
入力する時に関数名などがよくわからない場合、関数名の一部の文字や関連する文字を入力して F8 キーまたは右 Win キー(Apps キー)を押すと関係する関数名などが表示される。
例えば平方根の sqrt() 関数を入力したい時は sq と入力して F8 キーまたは平方入力して F8 キーを押すといくつかの入力候補が表示されるのでその中から選択する。
その他にカット(Ctrl+X),コピー(Ctrl+C) & ペースト(Ctrl+V) やアンドゥ(Ctrl+Z)、リドゥ(Ctrl+Y)が使用できる。

プログラムの実行

入力したプログラムの実行は F5 キーを押すかツールバーの右側の▶ボタンを押すと実行される。

```
a = 1;
b = a + 3;
println("a, b = ", a, ", ", b);
```

```
[09:06:28] Start : [test1]
a, b = 1, 4
[09:06:28] End : [test1]
```

プログラムの実行結果は開始時間と終了時間を加えて右上の出力ウィンドウに表示される。

プログラムの一時停止と中断

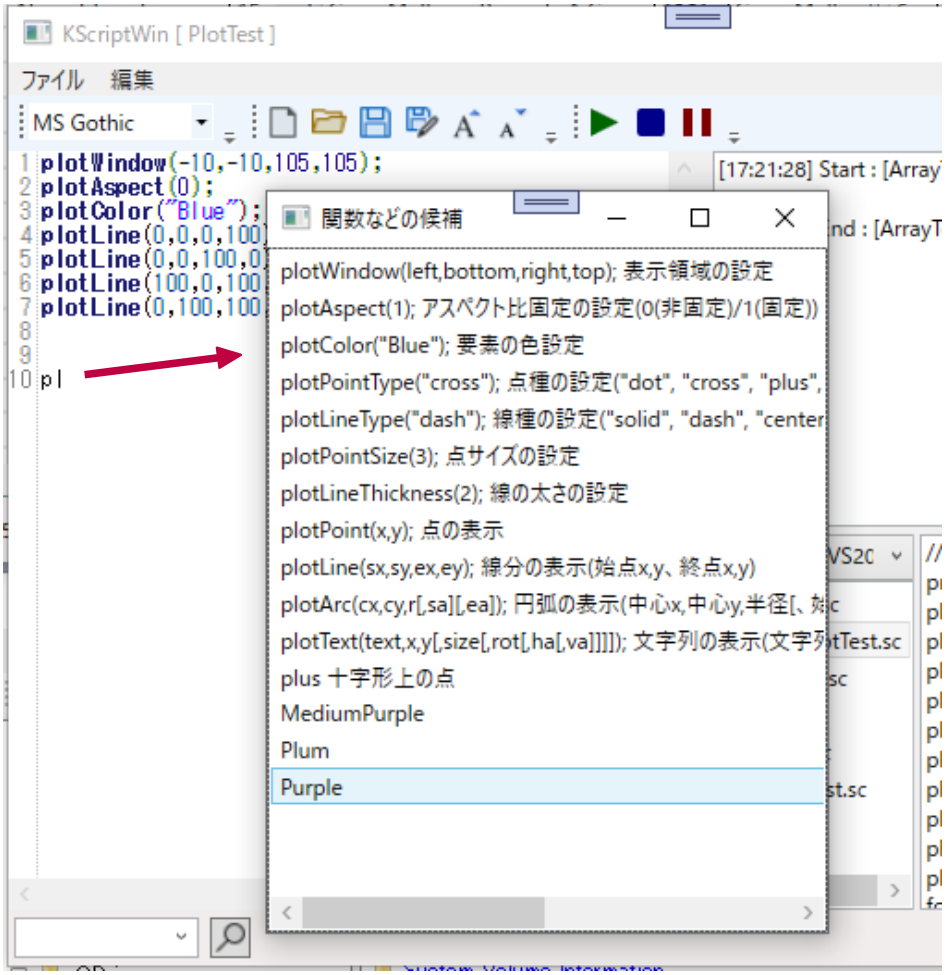
プログラムを実行して無限ループなどに入っ場合など処理を中断させた場合には、上部ツールバーの右側の中断ボタンまたは一時停止ボタン(Ctrl+D)を押して中断させることができる。
一時停止ボタンの場合は再度ボタンを押すと処理が再開される。

ファイル参照

他のファイルを参照しながらプログラムを入力する場合には、右下のファイル参照ウィンドウにファイルを表示して参照することができる。
ファイの選択はファイル参照ウィンドウの左上の参照フォルダのコンボボックスに登録してあるディレクトリを選択しその下のファイルリストでファイルを選択して表示する。
参照フォルダにディレクトリを追加する時は参照フォルダのコンボボックスをマウスでダブルクリックするとフォルダの選択ダイヤログが表示されるのでそれでフォルダを選択して追加する。

入力候補

エディタウィンドウで入力した文字から検索した関数名や定数などを説明文と合わせて表示し、該当の選択して入力する。
検索対象文字は関数名だけではなく、説明文に含まれる文字も指定できる。
使用方法は検索したい文字列の入力途中で Apps(右側 Win キー)または F8 キーを押すと該当する関数名などが表示される。
下図では pl と入力した後に F8 キーを押すとメニューダイヤログが表示されこの中から選択する。



・#include 文がある場合

インクルードするファイルから関数名とコメントを取り込んで入力候補に追加される。
下記の場合は 「roundTop(x) 最上位の桁で丸める」
新規に #include を追加した場合にはスクリプト文を再読み込みすると入力候補に出てくる。

```
// 最上位の桁で丸める
roundTop(x) {
    w = floor(log(x));
    return round(x, 10^w);
}
```

3. スクリプト文

C 言語風スクリプト言語

変数 ：型指定なし 実数または文字列(ダブルクォーテーション(")で囲む)
a = 12.5;
a = "abcd";

配列 ：1 次元配列 a[2] = 25;
a[m] = "abcd";
fruit["apple"] = 50;

2 次元配列 a[1,2] = 30;
a[m,n] = 10:

配列一括設定 a[] = { 1, 1, 5, 8 };
a[] = b[];
a[,] = { {1, 2, 3 }, { 2, 3, 4 } };
a[,] = b[,];
a[m,] = { 1, 1, 5, 8 };
a[m,] = b[];
a[m,] = b[n,];

グローバル変数 : 変数名の先頭が “g_”で始まるものはグローバル変数でそれ以外がローカル変数になる。
g_abcd = 2.5;

識別子 : = (代入文) , ; (文末)

演算子 : * , - , * , / , %(剰余) , ^(べき乗) , ++ , -- , += , -= , *= , /= , ^=

関係演算子 : == , != , < , > , <= , >=

論理演算子 : && , || , !

制御文 : 代入文 : 変数/配列 = 変数|配列|値|文字列|関数|数式 ;
if 文 : if (条件) { 実行文 } [else { 実行分 }]
while 文 : while (条件) { 実行文 }
for 文 : for (初期値; 条件; 更新処理) { 実行文 }
return 文 : return 戻り値 ;
break 文 : break;
continue 文 : continue;
exit 文 : exit;

関数定義 : 関数名(引数 1, 引数 2...) { 実行分; ... return 戻り値; }

プログラムの構造

```
main() {                    main 関数(省略可)
    a = 3;                   代入文
    b = 2;
    c = add(a, b);           関数呼び出し
    println(a + b, " ", c);
}
add(c, d) {                 関数定義
    return c + d;
}
```

4. スクリプト文の書き方

スクリプト文の書き方

スクリプト文(プログラム) は一つの main 関数と複数の sub 関数で構成される。
main 関数の main() { } は省略することもできる。
その中で使用する変数のうちローカル変数はその関数内でしか使用できないがグローバル関数(g_で始まる変数)はすべての関数を通して使用できる。

```
//    main 関数
main() {
    c = sub(a,b);
    println(c);
}
```

```
//    main 関数
//    main() { } は省略可能
c = sub(a,b);
println(c);
```

```
//    関数
sub(d, e) {
    return d - e;
}
```

:

```
//    関数
add(d, e) {
    return d + e;
}
```

関数内の文は基本的にはセミコロン(;)で終了する。

```
a = b + c;
```

{ } で囲まれた所には複数の文を書くことができる。

・{ } なしは 1 文のみ

```
if (a < b)
```

```
    println(a);
else
    println(b);
```

・{ } ありは複数文

```
if (a < b) {
    a = a +2;
    println(a);
} else {
    b = b -2;
    println(b);
}
```

コメント

// から行末までをコメントアウトする

/* から */ もコメントアウトするこの場合は複数行になってもかまわない

１行のみコメント化

```
//  コメント
a = a + 2;
```

以下はすべてコメント

```
/*
a = a + 2;
b = a * 4;
*/
```

ローカル変数

値を格納するもので特に型指定の必要はなくて値としては実数と文字列が使える

数値と文字列を足すと文字列として結合する(日本語可))

```
a = 3;
b = "abcd";
c = a + b;
print(c);    →  "3abcd"

住所 = "東京都";
println(住所);          →  東京都
```

グローバル変数

変数名の先頭が “g_” で始まるものはグローバル変数となる

```
g_val = 1;
val = 1;
inc();
println(g_val, ", ", val);      →        2, 1

inc() {
    g_val++;
    val++;
```

```
}
```

配列

複数のデータをまとめて扱うもので[ ]の中の数値を変えて値を格納する
配列のインデックスには文字列も使用できる。
配列の大きさは設定する必要はなく、配列にデータを入れることによって追加されていく。

```
a[0] = 10;
a[1] = 20;
a[2] = 30;
a["apple"] = 10;
```

値をまとめて設定するときは { } で括ってカンマ区切りで入力する
配列のインデックスには文字列も使用でき、値も文字列と数値の混在も可能
一括で値を設定する時は配列のインデックスは 0 から始まる

```
a[] = { 10, 20, 30 };
a[] = { “name”, 18, “men”};
println(a[0], ”,”, a[1], ”,”, a[2]);        → name, 18, men
```

2 次元配列

2 次元配列に一括でデータを設定する場合には { } を 2 重に囲って行う。

```
a[,] = { { 1, 2, 3}, {10, 20, 30} };
println(a[1,0], " , ", a[1,1], " , ", a[1,2]);        → 10, 20, 30
```

行ごとに設定する場合

```
a[2,] = {  100, 200, 300 };
for (i = 0; i < count(a[2,]); i++)
        print(a[2,i], " , ");                         → 100, 200, 300
```

配列のインデックスに文字列を使用

```
person["yamada",] = { "tarou", 68, "men" };
println(person["yamada",0], ",", person["yamada",1], ",", person["yamada",2],);
                                                      → tarou, 68, men

person["山田", "名"] = "太郎";
person["山田", "年齢"] = 68;
person["山田", "性別"] = "男";
println(person["山田", "名"]);                        →  太郎

item[] = { "名", "年齢", "性別" };
for (i = 0; i < count(item[]); i++) {
        print(person["山田",item[i]], " ");
}                                                     →  太郎 68 男
```

演算子

数式を処理するための演算子

+ ：加算　　a = 2 + 3;
- ： 減算　　a = 3 – 2;
* ： 乗算　　a = 2 * 3;
/ ：　除算　　a = 2 / 3;
% ： 剰余　　a = 2 % 3;　　(2 / 3 の余り→ 2)
^ ：べき乗　　a = 2 ^ 3;　　(2^3 → 8)

複合演算子

2 つの演算子を組み合わせて演算をおこなう

+= ： a += 3;　(a = a + 3 と同じ)
-= ： a -= 3;　(a = a - 3 と同じ)
*= ： a *= 3;　(a = a * 3 と同じ)
/= ： a /= 3;　(a = a / 3 と同じ)
%= ： a %= 3; (a = a % 3 と同じ)
^= ： a ^= 3;　(a = a ^ 3 と同じ)
++ ：　a++;　　インクリメント(a = a + 1 と同じ)
-- ：　a--;　　デクリメント(a = a – 1 と同じ)

比較演算子

条件文の中で 2 つの値を比較する演算子(if 文, while 文, for 文の条件文で使用)
条件文の中に数値や文字が設定された場合は常に true として処理される。

== ：等しい
!= ： 等しくない
< ： 小さい
> ： 大きい
<= ： 小さいか等しい
>= ： 大きいか等しい

```
if (a == b) print(a);
```

条件文の中に数値や文字が設定された場合は常に true として処理される。

```
if (1) println("true"); else println("false");    → true
```

数値の前に！をつけると false となる

```
if (!1) println("true"); else println("false");    →  false
```

論理演算子

比較演算子の結果を演算する

&& ：a && b　a かつ b
||　：a || b　　a または b
!　：!a　　a ではない

※ 論理演算子と比較演算子が混在する場合、優先順位がつかないので括弧をつけて優先順位をつける
a > 10 && a < 20 → (a > 10) && (a < 20)

```
a = 5;
if ((4 < a) && (a < 8))
   println("5 以上 7 以下");
else
   println("4 以下か 8 以上");
```

→ 5 以上 7 以下

制御文

if 文 ：条件によって分岐させる

実行分が複数の場合は { } で囲む必要があるが、単独の場合は省略も可能

if (条件) { 実行文; } [else { 実行文; }]

```
if (a > b) println(a);
```
a より b が大きければ a の値を表示

```
if (a > b) println(a);
else println(b);
```
a より b が大きければ a の値を表示
そうでなければ b の値を表示

```
if (a > b) {
   println(a);
} else if ( a == b) {
   println(a);
   println(b);
} else {
   println(b);
}
```
a より b が大きければ a の値を表示
a と b が同じときは a と b の値を表示
そうでなければ b の値を表示

while 文 ：条件に入っている間繰り返す

while (条件) { 実行文; }

```
n = 0;
while (n < 10) {
   n = n + 1;
   print(n, " ");
}
```

→ 1 2 3 4 5 6 7 8 9 10

```
//      配列の中から s 以上の数を検索
a[] = { 6, 9, 12, 7, 2, 23, 10, 4 };
s = 15;
n = 0;
while ((n < count(a[])) && (a[n++] < s)) ;
println(n, " ", a[n]);
```

→ 6 10　　(検索された配列の次の値を表示)

```
n = -1;
while ((n < count(a[])) && (a[++n] < s)) ;
println(n, " ", a[n]);
```

→ 5 23　　(検索された値を表示)

for 文　: 初期値、条件、増分を指定して繰り返す

for (初期値; 条件; 増分) { 実行文; }

```
for (i = 0; i < 10; i++)
   print(i, " ");
```

→　　0 1 2 3 4 5 6 7 8 9

```
for (i = 0; i < 10; i += 2)
   print(i, " ");
```

増分を 2 とした場合

→　　0 2 4 6 8

return 文 : 処理を中断して呼出し元に戻る

return (戻り値);

```
add(a,b) {
   return a+b;
}

max(a,b) {
   if (a>b)
      return a;
   else
      return b;
}
```

break 文 : ループの中断

```
n = 0;
while (1) {                    //  条件文に数値を入れると常に条件が成立
   if(10 <n) break;
   n++;
}
```

continue 文　: ループの先頭に戻る

```
n = 0;
while (n++ < 10) {
   if(n < 6) continue;         6 未満の場合は表示しない
   print(n," ");
}
```

→ 6 7 8 9 10

exit 文 : プログラムを終了させる

```
a= 1;
b = a+2;
exit;                    ここでプログラムを打ち切る
println(b);
```

関数の定義

複数の文をまとめて実行する仕組み(日本語可)

関数名(引数 1, ...) { 実行文; ... return 返値; }

```
add(a, b) {
    c = a + b;
    return c;
}
```

呼出し側

```
s = 2;
t = 3;
sum = add(s, t);
```

引数に配列を使う場合

```
sum(a[]) {
    size = arraySize(a[]);
    sum = 0;
    for (i = 0; i < size; i++) {
        sum += a[i];
    }
    return sum;
}
```

呼出し側

```
array[] = { 1, 2, 3, 4, 5 }
total = sum(array[]);
```

ディレクティブ

#include　指定したスクリプトファイルを追加読み込みをおこなう。
作成した汎用の関数を使用するときに使う。

#include “スクリプトファイル名”;

```
#include “scriptLib.sc”;
```

フルパスを指定しない場合には実行するスクリプトファイルと同じディレクトリが指定される。

数式処理以外の関数

print(a, b, "\n");	: コンソールに文字出力(Win 版では出力ウィンドウに出力)
println(a, b);	: print と同じで末尾に改行コードがつく
a = input();	: コンソールからの文字列入力(Win では使用不可、代わりに inputBox() を使用)
a = inKey();	: キー入力(1 文字)
sleep(n);	: 指定時間停止(約 n m 秒)

cmd(command);　　: Windows コマンドの実行
　　ex. cmd(“dir”);, cmd("KScriptWinManual.pdf");

配列関数(array.xxxx())

c = array.contain(a[2]);　　: 配列の有無(0:なし 1:あり)
size = array.count(a[]);　　: 配列のサイズ
size = array.count(a[,]);　　: 2 次元配列の全体の数
size = array.count(a[1,]);　　: 2 次元配列(2 行目)の列数
max = array.max(a[]);　　: 配列の最大値
min = array.min(a[,]);　　: 配列の最小値
sum = array.sum(a[]);　　: 配列の合計
ave = array.average(a[,]);　　: 配列の平均
vari = array.variance(a[]);　: 分散
std = array.stdDeviation(a[]);　　: 標準偏差
cov = array.covariance(a[], b[]);　: 共分散
corr = array.corrCoeff(x[],y[]);　　: 配列の相関係数
array.clear(a[]);　　: 配列をクリア
array.remove(a[],st[,ed]);　　: 配列の要素削除(1 次元のみ)
array.sort(a[]);　　: 配列のソート(1 次元のみ)
array.reverse(a[]);　　: 配列の逆順化(1 次元のみ)
c[] = array.concat(a[],b[]);　　: 配列の結合

マトリックス関数(matrix.xxxx(), 2 次元配列のみ)

a[,] = matrix.unit(size);　　: 単位行列
d[,] = matrix.transpose(a[,]);　　: 転置行列
d[,] = matrix.multi(a[,], b[,]);　　: 行列の積
d[,] = matrix.add(a[,], b[,]);　　: 行列の和
d[,] = matrix.inverse(a[,]);　　: 逆行列 a^-1
d[,] = matrix.copy(a[,]);　　: 行列のコピー

ダイヤログ関数

a = inputBox();　　: 文字入力ダイヤログ
messageBox(outString[, title]);　: 文字列のダイヤログ表示
n = menuSelect(menu[],title);　　: 選択メニューを表示して選択項目 No を返す

CAD 関数(cad.xxxx())

cad.disp();　　再表示
cad.dispFull();　　全体表示
cad.setColor(\"Blue\");　　色の設定
cad.setLineType(\"dash\");　　線種の設定(\"solid\", \"dash\", \"center\", \"phantom\")
cad.setLineThickness(2);　　線の太さの設定

cad.setTextPara(size,rotate,HorizontalAlignment,VerticalAlignment,LinePitchRate,
FontStyle,FontWeight): 文字パラメータの設定
cad.line(sx,sy,ex,ey/sp[],ep[]/plist[,]); 線分を作成
cad.arc(cp[],r[,sa[,ea]]); 円/円弧の作成
cad.arc(sp[],mp[],ep[]/plist[]); 三点円弧の作成
cad.circle(cp[],r); 半径指定円の作成
cad.circle(x0,y0,x1,y1,x2,y2/sp[],mp[],ep[]/plist[,]); 三点円の作成
cad.ellipse(cp[],rx,ry,rotate,sa,ea): 楕円子の作成
cad.oval(plist[,]/p0[],p1[]..../x0,y0,x1,y1...): 楕円の作成
cad.polyline(p[,]/p0[],p1[],p2[]..../x0.y0,x1,y1...); ポリラインの作成
cad.polygon(p[,]/p0[],p1[],p2[]..../x0.y0,x1,y1...); ポリゴンの作成
cad.text([pos[]/x,y],text,size,rotate,horizontal,vertical): 文字列の作成
cad.arrow(plist[,],ArrowSize/p0[],p1[],p2[]...,ArrowSize/x0,y0,x1,y1...,ArrowSize): 矢印の作成
cad.label(plist[,],TextSize/p0[],p1[],p2[]...,TextSize/x0,y0,x1,y1...,TextSize): ラベルの作成

数式で使う関数

pi = PI; : 円周率(3.14..)
e = E; : 自然対数の底(2.71..)
rad = RAD(deg); : 度をラジアンに変換
deg = DEG(rad); : ラジアンを度に変換
hour = deg2hour(deg); : 度を時単位に変換(360° →24 時間)
deg = hour2deg(hour); : 時単位を度に変換(1 時間 → 15°)
hour = rad2hour(rad); : ラジアンを時単位に変換(PI → 12 時間)
rad = hour2rad(hour); : 時単位をラジアンに変換(12 時間 → PI)
a = mod(x,y); : 剰余(x / y の余り)
a = pow(x,y); : 累乗 (x^y)
a = max(x,y); : x と y の大きい方
a = min(x,y); : x と y の小さい方
a = combi(n,r); : 組合せの数(nCr)
a = permu(n,r); : 順列の数(nPr)
a = sin(x); : 正弦
a = cos(x); : 余弦
a = tan(x); : 正接
a = asin(x); : 逆正接
a = acos(x); : 逆余弦
a = atan(x); : 逆正接
a = atan2(x,y); : 逆正接
a = sinh(x); : 双曲線正弦
a = cosh(x); : 双曲線余弦
a = tanh(x); : 双曲線正接
a = asinh(x); : 逆双曲線正弦
a = acosh(x); : 逆双曲線余弦

a = atanh(x); : 逆双曲線正接
a = exp(x); : e の累乗
a = ln(x); : e を底とする自然対数
a = log(x); : 10 を底とする対数
a = log(x,y); : x を底とする y の対数
a = sqrt(x); : 平方根
a = abs(x); : 絶対値
a = ceil(x); : 切上げ(x 以上で最小の整数値)
a = floor(x); : 切捨て(小数点以下の数の内最大の整数値)
a = round(x); : 四捨五入(もっとも近い整数値)
a = trunc(x); : 浮動小数点の整数部
a = sign(x); : 符号示す値(1/0/-1)
a = round(x,y); : x を y の倍数に丸める
a = equals(x,y); : 等価判定 x == y ⇒ 1, x != y ⇒ 0
a = lt(x,y); : 大小判定(less than) x > y ⇒ 1,以外は 0
a = gt(x,y); : 大小判定(greater than) x < y ⇒ 1,以外は 0
a = compare(x,y); : 大小判定 x > y ⇒ 1,x == y ⇒ 0, x < y ⇒ -1
a = deg2dms(x); : 度(ddd.dddd) → 度分秒(ddd.mmss)
a = dms2dig(x); : 度分秒(ddd.mmss) → 度(ddd.dddd)
a = hour2hms(x); : 時(hh.hhhh) → 時分秒(hh.mmss)
a = hms2hour(x); : 時分秒(hh.mmss) → 時(hh.hhhh)
a = fact(x); : 階乗 (a = x * (x-1) * … * 2 * 1)
a = fib(x); : フィボナッチ数列 (Fn = Fn-1 + Fn-2 , F2 = 1, F1 = 0)
a = gcd(x,y); : 最大公約数
a = lcm(x,y); : 最小公倍数
a = JD(y,m,d); : 西暦年月日からユリウス日を求める
a = MJD(y,m,d); : 西暦年月日から準ユリウス日を求める
a = JD2Date(x); : ユリウス日を年月日に変換して yyyymmdd の実数にする
a = sum(f([@]),n,k); : 級数の和 n から k まで連続し値を計算式 f([@])で演算した値の合計を求める(sum([@]*[@],1,10) → 1*1+2*2+...10*10 → 385)
a = sum(f([@]),n1,n2...nm);: 級数の和 n1 から nm まで値を計算式 f([@])で演算した値の合計を求める (sum([@]^2,1,2,3,4,5) → 1^2+2^2+3^2+4^2+5^2 → 55)
a = product(f([@]),n,k); : 級数の積 n から k まで連続し値を計算式 f([@])で演算した値の積を求める (product([@],1,5) → 120)
a = product(f([@]),,n1,n2...nm); : 級数の積 n1 から nm まで値を計算式 f([@])で演算した値の積を求める (product([@]^2,1,2,3,4,5) → 14400)
a = repeat(f([@],[%]),i,n,k);: 計算式の[@]に n から k まで入れて繰返す,[%]に計算結果が入る,i は[%]の初期値 (repeat([%]*[@],10,2,6) → ((((10*2)*3)*4)*5)*6 → 7200)
例 : 元本 1 万円を年利 2%で 5 年預けた場合の金額
repeat([%]*1.02,10000,1,5) ⇒
((((10000*1.02)*1.02)*1.02)*1.02))*1.02 = 11040.808

5. 配列関数

array. で始まる関数は配列に関する関数
使用できる配列の種類はインデックスが数値で 0 以上のものに限る。

関数の種類

a = array.contains(c[2]);	配列内で存在を調べる関数 (a = 0 :存在しない a = 1 : 存在する
size = array.count(a[]);	1 次元配列のサイズ
size = array.count(a[,]);	2 次元配列の全体サイズ
size = array.count(a[m,]);	2 次元配列 m 行目のサイズ(列数)
size = array.count(a[,n]);	2 次元配列 n 列目のサイズ(行数)
c[,] = array.concat(a[,],b[,]);	配列同士を結合して新しい配列にする
array.clear(a[]);	配列データをクリア
array.remove(a[],start[,end]);	１次元配列要素を範囲指定で削除
array.squeeze(a[]);	配列の未使用データを削除圧縮
array.sort(a[]);	１次元配列のソート
array.sort(a[,n]);	２次元配列を n 列でソート
array.reverse(a[]);	１次元配列の順番を逆にする
array.reverse(a[,]);	２次元配列の行の順番を逆なする
array.reverse(a[,],1);	２次元配列の列の順番を逆なする(2 番目の引数が 1 の時)
max = array.max(a[]);	1/2 次元配列の最大値を求める
max = array.max(a[m,]);	2 次元配列の m 行目の最大値を求める
min = array.min(a[]);	1/2 次元配列の最小値を求める
min = array.min(a[m,]);	2 次元配列の m 行目の最小値を求める
sum = array.sum(a[]);	1 次元配列の合計
ave = array.average(a[]);	1 次元配列の平均
ave = array.average(a[,]);	２次元配列の全体の平均
ave = array.average(a[m,]);	２次元配列の m 行目の平均
vari = array.variance(a[]);	配列の分散
std = array.stdDeviation(a[]);	配列の標準偏差
cov = array.covariance(a[], b[]);	共分散
coor = array.corrCoeff(x[],y[]);	配列の相関係数

6. マトリックス関数

matrix. で始まる関数は行列の演算処理を行う関数

関数の種類

matrix.unit(size);　　単位行列(2 次元)の作成

```
a[,] = matrix.unit(3);
```

→

```
1 0 0
0 1 0
0 0 1
```

matrix.transpose(a[,]);　　転置行列(2 次元行列 A の転置(A^T)　)

```
a[,] = {
    { 1, 2, 3 },
    { 4, 5, 6 },
    { 7, 8, 9 }
}
d[,] = matrix.transpose(a[,]);
```

→

```
1 4 7
2 5 8
3 6 9
```

matrix.multi(a[,], b[,]);　　行列の積 AxB (c[,]=…

```
b[,] = {
    { 1, 2 },
    { 4, 5 },
    { 7, 8 }
}
d[,] = matrix.multi(a[,], b[,]);
```

→

```
30   36
66   81
102 126
```

matrix.add(a[,], b[,]);　　行列の和 A+B

```
e[,] = {
    { 10, 20, 30 },
    { 40, 50, 60 },
    { 70, 80, 90 }
}
d[,] = matrix.add(a[,], e[,]);
```

→

```
11 22 33
44 55 66
77 88 99
```

matrix.inverse(a[,]);　　逆行列 A^-1

```
a[,] = {{1,2},{3,4}};
b[,] = matrix.inverse(a[,]);
```

→

```
-2     1
 1.5 -0.5
```

matrix.copy(a[,]);　　行列のコピー

```
d[,] = matrix.copy(a[,]);
```

→

```
1 2
3 4
```

7. CAD 機能

設定関数

要素作成関数

線分の表示

線分を表示する場合、その属性として 色(plot.Color())、線種(plot.LineType())、太さ(plot.Thickness())が設定でき、次に設定されるまでその属性が有効になる。
ちなみに色はすべての要素に、線種と太さは円弧にも有効である。
次の場合は Blue、破線 で線分が表示される。
線分の色は 「色」と入力した後 F8 キーまたは右 Win キー を押すと入力候補が表示され、線種も同様に入力候補を出すことができる。
ちなみに線種は solid (実線)、dash(破線)、center(一点鎖線)、phantom(2 点鎖線)の 4 種類がある。

```
plot.Color("Blue");
plot.LineType("dash");
plot.Line(20,20,80,80);
```

円弧の表示

円弧を表示するときは下記の書式にしたがって行う。
開始角と終了角の単位はラジアンで省略可能である。
その場合、開始角 = 0、終了角 -=2*PI となり円で表示される。

plot.Arc(x 座標, y 座標, 半径, 開始角, 終了角);

文字列の表示

文字列を表示する場合は次のような書式でおこなう。
・文字高さ　　ワールド座標で設定されるので Window の大きさを変えるとそれに合わせて変わる
・回転角　　文字列の回転角を設定で 単位はラジアンで設定する。
・水平アライメント　文字列の水平方向の基準位置 0:左 1:中央 2:右
・垂直アライメント　文字列の垂直方向の基準位置 0:上 1:中央 2:下

※水平垂直のアライメントを省略すると基準位置は左上、回転角を省略すると回転角は 0°となる。

plot.Text(“文字列”, x 座標, y 座標, 文字高さ, 回転角, 水平アライメント, 垂直アライメント);

```
fontSize = 5;
rotate = RAD(0);
plot.Text("left",10,10,fontSize,rotate,2,2);
```